

SpaceInvader

Marco Geisler

July 6, 2026

1 Einleitung

Damit wir unser Spiel erstellen können, müssen wir als erstes eine Datei anlegen. Diese Datei kann einen beliebigen Namen haben, muss jedoch auf `.py` enden. In unserem Beispiel nennen wir unsere Datei `main.py`.

1.1 Der Import-Befehl

Der Befehl zum Importieren von Pygame lautet:

```
1 import pygame
```

Dieser Befehl lädt das Pygame-Modul, sodass dessen Funktionen im Skript genutzt werden können.

1.2 Warum muss Pygame importiert werden?

Python verfügt nicht standardmäßig über die Pygame-Bibliothek, daher muss sie explizit importiert werden. Dies stellt sicher, dass alle relevanten Funktionen und Klassen verfügbar sind. Nach dem Import kann man Pygame mit folgendem Befehl initialisieren:

```
1 pygame.init()
```

Die Funktion `pygame.init()` bereitet alle Module von Pygame vor, sodass sie verwendet werden können.

1.3 Erstellung eines Spielfensters

Ein Spielfenster kann mit der Funktion `pygame.display.set_mode()` erstellt werden:

```
1 screen = pygame.display.set_mode((800,600))
```

Diese Funktion erzeugt ein Fenster mit einer Breite von 800 Pixeln und einer Höhe von 600 Pixeln. Die zurückgegebene Variable `screen` stellt die Zeichenfläche dar, auf die später Grafiken oder Objekte gezeichnet werden können. Das Spielfenster bleibt jedoch nur für eine kurze Zeit aktiv, wenn wir unser Programm jetzt starten, weil der Code von unten nach oben ausgeführt wird. Um dieses Problem zu beheben, können wir wie folgt vorgehen.

1.4 Ereignisschleife und Beenden des Fensters

Ein zentrales Konzept in Pygame ist die Ereignisschleife, die kontinuierlich ausgeführt wird, solange das Spiel läuft. Diese Schleife stellt sicher, dass das Fenster auf Benutzereingaben reagiert und nicht sofort nach dem Erstellen geschlossen wird. Eine typische Ereignisschleife sieht folgendermaßen aus:

```
1 running = True
2 while running:
3     for event in pygame.event.get():
4         if event.type == pygame.QUIT:
5             running = False
```

Die Variable `running` wird am Anfang auf `True` gesetzt, damit die Schleife gestartet wird. Innerhalb der Schleife ruft `pygame.event.get()` alle aktuellen Ereignisse ab, die vom Benutzer oder dem System ausgelöst wurden (ein Ereignis kann beispielsweise das Drücken einer Taste oder das Betätigen einer Maus sein). Diese Ereignisse werden in einer `for`-Schleife durchlaufen und überprüft. Wenn ein Ereignis

vom Typ `pygame.QUIT` erkannt wird – etwa wenn der Benutzer das Fenster schließt –, wird die Variable `running` auf `False` gesetzt. Dadurch wird die Schleife im nächsten Durchlauf beendet, und das Programm kann ordnungsgemäß beendet werden. Ohne eine solche Schleife würde das Fenster nur für einen kurzen Moment erscheinen und sofort wieder geschlossen werden.

1.5 Titel, Icon und Hintergrund

1.5.1 Titel und Icon

Ein Pygame-Fenster kann mit einem individuellen Titel und Icon versehen werden, um das Erscheinungsbild des Spiels zu personalisieren. Dies geschieht mit den folgenden Befehlen:

```
1 # Titel und Icon
2 pygame.display.set_caption("Space Invaders")
3 icon = pygame.image.load('ufo.png')
4 pygame.display.set_icon(icon)
```

Mit `pygame.display.set_caption()` wird der Fenstertitel gesetzt, in diesem Fall auf *Space Invaders*. Dies beeinflusst die Titelleiste des Spielfensters und sorgt für eine bessere Wiedererkennbarkeit.

Das Icon des Fensters wird mit `pygame.display.set_icon()` gesetzt. Dafür wird zunächst das Bild mit `pygame.image.load()` geladen und dann als Icon registriert. Das verwendete Beispielbild `ufo.png` stammt von der Webseite [Flaticon](#), wo viele lizenzfreie Icons zu finden sind. Ein passendes Icon trägt zur Ästhetik des Spiels bei und hilft, es visuell ansprechender zu gestalten. Unser Icon haben wir mit der Größe 32x32 Pixel heruntergeladen. Die Bilddatei muss in den gleichen Ordner wie unsere Python-Datei gespeichert werden.

1.5.2 Hintergrundfarbe mit RGB setzen

Die Hintergrundfarbe des Pygame-Fensters kann mit der Funktion `screen.fill()` geändert werden. Dabei wird ein Farbwert im RGB-Format angegeben:

```
1 # RGB
2 screen.fill((255, 0, 0))
3 pygame.display.update()
```

Die Funktion `screen.fill((255, 0, 0))` füllt das gesamte Fenster mit der angegebenen Farbe. Die Werte in den Klammern repräsentieren die drei Farbkanäle: Rot, Grün und Blau. In diesem Fall bedeutet `(255, 0, 0)`, dass das Fenster vollständig rot eingefärbt wird, da der Rot-Kanal auf den maximalen Wert von 255 gesetzt ist, während Grün und Blau auf 0 stehen.

Damit die Änderung sichtbar wird, muss `pygame.display.update()` aufgerufen werden. Diese Funktion aktualisiert das Fenster und zeigt die neue Hintergrundfarbe an. Ohne dieses Update würde die Änderung nicht auf dem Bildschirm erscheinen. Außerdem muss dieser Code-Bereich in unserer Ereignisschleife stattfinden (damit ständig ein Update ausgeführt werden kann).

1.6 Spieler-Setup

Ein zentrales Element des Spiels ist der Spielercharakter, der als Bild geladen und auf dem Bildschirm positioniert wird. Dies geschieht mit den folgenden Befehlen:

```
1 # Player
2 playerImage = pygame.image.load('space-invaders.png')
3 playerX = 370
4 playerY = 480
```

Das Spielerbild `space-invaders.png` wird mit `pygame.image.load()` geladen. Dieses Bild stammt von der Webseite [Flaticon](#) und kann dort mit der Suchanfrage "space-invaders" gefunden werden. Heruntergeladen soll es mit einer Auflösung von 64x64 Pixeln werden. Die Bilddatei muss in den gleichen Ordner wie unsere Python-Datei gespeichert werden.

Die Koordinaten `playerX = 370` und `playerY = 480` legen die Startposition des Spielers fest. Das Spielfenster hat eine Breite von 800 Pixeln und eine Höhe von 600 Pixeln:

- Die horizontale Position 370 wurde gewählt, da das Spielerbild mittig erscheinen soll. Da die Breite des Fensters 800 Pixel beträgt, liegt die Mitte bei $800 / 2 = 400$. Da das Bild selbst eine gewisse Breite

hat, wurde ein leicht kleinerer Wert 370 gewählt, um die Mitte auszugleichen.

- Die vertikale Position 480 wurde gewählt, damit der Spieler sich nahe am unteren Rand des Fensters befindet. Da das Fenster 600 Pixel hoch ist, liegt der untere Bereich nahe bei $600 - 120 = 480$, wobei die 120 Pixel einen Abstand zum unteren Rand einhalten.

Zusätzlich gibt es die Funktion `player()`, die den Spieler auf dem Bildschirm zeichnet:

```
1 def player():
2     screen.blit(playerImage, (playerX, playerY))
```

Die Funktion `blit()` zeichnet das Spielerbild an der angegebenen Position. Damit wird sichergestellt, dass der Spieler sichtbar ist und korrekt auf dem Bildschirm dargestellt wird. Diese Funktion darf erst nach der Färbung des Hintergrundes aufgerufen werden, ansonsten würde unser Raumschiff vom Hintergrund übermalt werden.

Code nach Kapitel 1

```
1 import pygame
2
3 # wir initialisieren (erstellen) das Spiel.
4 pygame.init()
5
6 # erstellen des Bildschirms
7 screen = pygame.display.set_mode((800,600))
8
9 # Titel and Icon
10 pygame.display.set_caption("Space Invaders")
11 icon = pygame.image.load('ufo.png')
12 pygame.display.set_icon(icon)
13
14 # Player
15 playerImage = pygame.image.load('space-invaders.png')
16 playerX = 370
17 playerY = 480
18
19 def player():
20     screen.blit(playerImage, (playerX, playerY))
21
22 # Gameloop
23 running = True
24 while running:
25     for event in pygame.event.get():
26         if event.type == pygame.QUIT:
27             running = False
28
29     #RGB
30     screen.fill((0, 0, 0))
31
32     player()
33     pygame.display.update()
```

2 Interaktion mit dem Spiel

Bisher können wir nur ein statisches (festes) Fenster mit unterschiedlichen Bildern und Farben erstellen. Damit daraus jetzt ein Spiel werden kann, müssen wir irgendwie mit dem Spiel interagieren können. Dafür sorgen wir als erstes dafür, dass wir unser Raumschiff bewegen können.

2.1 Spieler nach rechts bewegen

Um den Spieler flexibler zu platzieren und bewegen zu können, können wir die Funktion `player()` so anpassen, dass sie nun Parameter `x` und `y` akzeptiert:

```
1 def player(x, y):  
2     screen.blit(playerImage, (x, y))
```

Diese Änderung ermöglicht es, die Position des Spielers dynamisch zu steuern, indem beim Funktionsaufruf die aktuellen Koordinaten übergeben werden.

Ein Beispiel für die Bewegung des Spielers ist:

```
1 playerX += 0.1  
2 player(playerX, playerY)
```

Hier wird der Wert von `playerX` kontinuierlich um 0.1 erhöht, was dazu führt, dass sich der Spieler langsam nach rechts bewegt. Anschließend wird die Funktion `player()` mit den neuen Koordinaten aufgerufen, um die aktualisierte Position auf dem Bildschirm darzustellen. Analog können wir unseren Spieler nach links bewegen, wenn wir statt plus zu rechnen, minus rechnen. Das gleiche gilt für die Bewegung von oben nach unten, indem wir `playerY` ändern.

2.2 Spielerbewegung mit Tasteneingaben

Bisher wurde die Position des Spielers durch eine feste Erhöhung von `playerX` um 0.1 verändert. Dies wird nun durch eine dynamische Steuerung mittels Tasteneingaben ersetzt, weshalb wir unsere alte Logik mit der festen Erhöhung von `playerX` um 0.1 entfernen können.

Dafür wird die Variable `changeX` eingeführt, die die Änderung der X-Position speichert (die anderen Variablen sind bereits vorhanden):

```
1 # Player  
2 playerImage = pygame.image.load('space-invaders.png')  
3 playerX = 370  
4 playerY = 480  
5 changeX = 0
```

Nun können wir in unserer for-Schleife, die die Events kontrolliert, überprüfen, ob gewisse Tasten gedrückt sind:

```
1 for event in pygame.event.get():  
2     if event.type == pygame.KEYDOWN:  
3         if event.key == pygame.K_LEFT:  
4             changeX = -0.3  
5         if event.key == pygame.K_RIGHT:  
6             changeX = 0.3  
7     if event.type == pygame.KEYUP:  
8         if event.key == pygame.K_LEFT or event.key == pygame.K_RIGHT:  
9             changeX = 0
```

Jedes Ereignis in Pygame wird durch die Ereignisschleife mit `pygame.event.get()` erfasst. Hierbei prüft der Code den Typ des Ereignisses mittels `event.type`:

- `pygame.KEYDOWN`: Dieses Ereignis tritt auf, wenn eine Taste auf der Tastatur gedrückt wird.
- `pygame.KEYUP`: Dieses Ereignis tritt auf, wenn eine Taste losgelassen wird.

Innerhalb der `KEYDOWN`-Abfrage wird überprüft, welche Taste gedrückt wurde. Dies geschieht mit `event.key`:

- `pygame.K_LEFT`: Diese Konstante steht für die linke Pfeiltaste. Wenn diese gedrückt wird, setzt sich `changeX` auf -0.3, wodurch sich der Spieler nach links bewegt.
- `pygame.K_RIGHT`: Diese Konstante steht für die rechte Pfeiltaste. Wenn diese gedrückt wird, setzt sich `changeX` auf 0.3, wodurch sich der Spieler nach rechts bewegt.

Sobald die Taste losgelassen wird (**KEYUP**), überprüft das Programm, ob es sich um die linke oder rechte Pfeiltaste handelt. Falls ja, wird **changeX** auf 0 gesetzt, wodurch die Bewegung gestoppt wird.

Es ist auch möglich, andere Tasten zur Steuerung zu verwenden, indem man beispielsweise **pygame.K_A** für die linke Bewegung und **pygame.K_D** für die rechte Bewegung einsetzt. Dies könnte folgendermaßen angepasst werden:

```
1 if event.key == pygame.K_a:
2     changeX = -0.3
3 if event.key == pygame.K_d:
4     changeX = 0.3
```

Die eigentliche Aktualisierung der Position erfolgt mit:

```
1 playerX = playerX + changeX
2 player(playerX, playerY)
```

Hier wird **playerX** kontinuierlich mit **changeX** aktualisiert. Der Wert von **changeX** bestimmt dabei die Geschwindigkeit der Bewegung – je größer der Wert, desto schneller bewegt sich der Spieler.

Da die Bewegung jetzt durch die Variable **changeX** gesteuert wird, ist die frühere feste Erhöhung von **playerX** um 0.1 nicht mehr erforderlich und wurde entfernt.

2.3 Begrenzung der Spielerbewegung

Damit der Spieler sich nicht über die Grenzen des Spielfensters hinaus bewegt, wird die Position nach jeder Aktualisierung überprüft und begrenzt:

```
1 playerX = playerX + changeX
2 if playerX <= 0:
3     playerX = 0
4 elif playerX > 736:
5     playerX = 736
6 player(playerX, playerY)
```

Da das Spielfenster eine Breite von 800 Pixeln hat und der Spieler eine Breite von etwa 64 Pixeln besitzt, wird die rechte Begrenzung auf 736 gesetzt (800 - 64). Falls **playerX** kleiner oder gleich 0 ist, wird es auf 0 gesetzt, um zu verhindern, dass der Spieler aus dem linken Bildschirmrand verschwindet.

Code nach Kapitel 2

```
1 import pygame
2
3 # wir initialisieren (erstellen) das Spiel.
4 pygame.init()
5
6 # erstellen des Bildschirms
7 screen = pygame.display.set_mode((800,600))
8
9 # Titel and Icon
10 pygame.display.set_caption("Space Invaders")
11 icon = pygame.image.load('ufo.png')
12 pygame.display.set_icon(icon)
13
14 # Player
15 playerImage = pygame.image.load('space-invaders.png')
16 playerX = 370
17 playerY = 480
18 changeX = 0
19
20 def player(x, y):
21     screen.blit(playerImage, (x, y))
22
23 # Gameloop
24 running = True
25 while running:
26     for event in pygame.event.get():
27         if event.type == pygame.QUIT:
28             running = False
29
30     # uberprufe, ob beim Drucken der Tastatur die Linkspfeiltaste oder
31     # Rechtspfeiltaste gedruckt wird
32     if event.type == pygame.KEYDOWN:
33         if event.key == pygame.K_LEFT:
34             changeX = -0.3
35         if event.key == pygame.K_RIGHT:
36             changeX = 0.3
37     if event.type == pygame.KEYUP:
38         if event.key == pygame.K_LEFT or event.key == pygame.K_RIGHT:
39             changeX = 0
40
41     #RGB
42     screen.fill((0, 0, 0))
43
44     #Anderung der x-Koordinate übernehmen
45     playerX = playerX + changeX
46
47     #Grenzen uberprufen
48     if playerX <= 0:
49         playerX = 0
50     elif playerX > 736:
51         playerX = 736
52
53     player(playerX, playerY)
54     pygame.display.update()
```

3 Gegner

Wir können jetzt schon mit unserem Spiel interagieren, unser Spiel hat jedoch noch kein wirkliches Ziel. Um dies zu ändern, können wir Gegner hinzufügen.

3.1 Gegner-Setup

Ähnlich wie der Spieler benötigt auch der Gegner eine eigene Darstellung und Positionierung im Spiel. Die Implementierung folgt demselben Prinzip, das bereits für den Spieler verwendet wurde.

```
1 # Enemy
2 enemyImage = pygame.image.load('alien.png')
3 enemyX = 370
4 enemyY = 50
5 changeXenemy = 0
6 changeYenemy = 0
```

Das Bild für den Gegner `alien.png` wird mit `pygame.image.load()` geladen. Die Variablen `enemyX` und `enemyY` legen die Startposition des Gegners fest. Während der Spieler am unteren Rand beginnt, wird der Gegner weiter oben auf `enemyY = 50` positioniert.

Das verwendete Gegnerbild `alien.png` stammt von der Webseite [Flaticon](#). Dort kann es mit der Suchanfrage "space-invader enemy" gefunden werden.

Um den Gegner auf dem Bildschirm darzustellen, wird eine Funktion definiert, die exakt so funktioniert wie die `player()`-Funktion:

```
1 def enemy(x, y):
2     screen.blit(enemyImage, (x, y))
```

Diese Funktion nimmt die aktuellen Koordinaten des Gegners als Parameter und zeichnet ihn an der entsprechenden Stelle.

Der Funktionsaufruf:

```
1 enemy(enemyX, enemyY)
```

sorgt dafür, dass der Gegner bei jedem Frame neu an seiner Position gerendert wird. Das Konzept ist nahezu identisch zur Darstellung des Spielers, lediglich die Startposition unterscheidet sich.

3.2 Zufällige Platzierung des Gegners

Um das Spiel dynamischer zu gestalten, wird die Position des Gegners nicht mehr fest vorgegeben, sondern zufällig bestimmt. Dafür wird das `random`-Modul importiert, das es ermöglicht, Zufallszahlen zu generieren.

```
1 import random
2
3 ...
4
5 # Enemy
6 enemyImage = pygame.image.load('alien.png')
7 enemyX = random.randint(0, 736)
8 enemyY = random.randint(50, 150)
9 changeXenemy = 0
10 changeYenemy = 0
```

Hierbei werden die Startkoordinaten des Gegners mit `random.randint()` bestimmt:

- `enemyX = random.randint(0, 736)`: Der Gegner wird irgendwo auf der horizontalen Achse zwischen 0 und 736 Pixeln platziert. Da das Spielfenster 800 Pixel breit ist, kann der Gegner theoretisch an jeder möglichen Position innerhalb des Fensters erscheinen.

- `enemyY = random.randint(50, 150)`: Der Gegner erscheint zufällig zwischen 50 und 150 Pixeln in der Höhe. Diese Begrenzung stellt sicher, dass der Gegner nicht zu weit oben (außerhalb des Fensters) oder zu weit unten (zu nah am Spieler) erscheint.

Durch diese Anpassung wird das Spiel abwechslungsreicher, da der Gegner nicht immer an derselben Stelle startet, sondern sich mit jeder neuen Runde an einer zufälligen Position befindet.

3.3 Bewegung des Gegners

Bisher wurde der Gegner nur an einer zufälligen Position platziert, bewegte sich jedoch nicht. Um dem Spiel mehr Dynamik zu verleihen, wird der Gegner nun horizontal hin und her bewegt. Zusätzlich rückt er nach unten, wenn er den Bildschirmrand erreicht.

```
1 ...
2
3 changeXenemy = 0.3
4 changeYenemy = 40
5
6 ...
7
8 # Bewegung des Gegners
9 enemyX = enemyX + changeXenemy
10 if enemyX <= 0:
11     changeXenemy = 0.3
12     enemyY += changeYenemy
13 elif enemyX > 736:
14     changeXenemy = -0.3
15     enemyY += changeYenemy
```

Erklärung der Logik: - Die Variable `changeXenemy` steuert die horizontale Bewegung des Gegners. Mit einem Wert von 0.3 bewegt sich der Gegner nach rechts, mit -0.3 nach links. - Die Variable `changeYenemy` legt fest, um wie viele Pixel der Gegner nach unten rückt, sobald er den Rand erreicht. Die Bewegung folgt diesem Ablauf:

1. **Horizontale Bewegung:** Der Wert von `enemyX` wird kontinuierlich um `changeXenemy` erhöht oder verringert, wodurch sich der Gegner nach rechts oder links bewegt.
2. **Bildschirmrand-Erkennung:** Falls `enemyX` kleiner oder gleich 0 ist, bedeutet dies, dass der Gegner den linken Rand erreicht hat. - Nun wird `changeXenemy` auf 0.3 gesetzt, damit sich der Gegner wieder nach rechts bewegt. - Außerdem wird `enemyY` um `changeYenemy` erhöht, sodass der Gegner eine Reihe nach unten rückt. - Falls `enemyX` größer als 736 ist (die rechte Grenze, basierend auf einer Fensterbreite von 800 Pixeln und einer ungefähren Gegnerbreite von 64 Pixeln), passiert das Gleiche in umgekehrter Richtung: - `changeXenemy` wird auf -0.3 gesetzt, damit sich der Gegner wieder nach links bewegt. - Der Gegner rückt erneut um `changeYenemy` nach unten.

Durch diese Logik entsteht das typische Bewegungsmuster von Gegnerformationen, die in klassischen Spielen wie *Space Invaders* verwendet werden.

3.4 Hintergrundbild einfügen

Ein statischer Hintergrund macht das Spiel optisch ansprechender und sorgt für eine immersive Spielerfahrung. In Pygame kann ein Bild als Hintergrund geladen und bei jedem Frame gezeichnet werden.

```
1 # Hintergrund
2 background = pygame.image.load('background.png')
```

Mit `pygame.image.load()` wird das Bild `background.png` geladen. Dieses Bild sollte die gleiche Auflösung wie das Spielfenster haben (z. B. 800x600 Pixel), damit es das gesamte Spielfeld abdeckt.

Damit der Hintergrund im Spiel sichtbar ist, muss er in jeder Schleifeniteration neu gezeichnet werden:

```
1 # Background
2 screen.blit(background, (0,0))
```

Hier wird das Bild mit `blit()` auf den Bildschirm gezeichnet. Die Koordinaten (0,0) bedeuten, dass das Bild in der oberen linken Ecke des Fensters beginnt.

Warum muss der Hintergrund jedes Mal neu gezeichnet werden? Da Pygame den Bildschirm in jedem Frame aktualisiert, würden vorherige Zeichnungen verschwinden, wenn sie nicht erneut gezeichnet werden. Ohne das erneute Zeichnen würde das Fenster nach Bewegungen nur noch Fragmente von Objekten zeigen. Daher ist es notwendig, den Hintergrund in jedem Frame neu zu rendern, bevor die Spielfiguren gezeichnet werden.

Code nach Kapitel 3

```
1 import pygame
2 import random
3
4 # wir initialisieren (erstellen) das Spiel.
5 pygame.init()
6
7 # erstellen des Bildschirms
8 screen = pygame.display.set_mode((800,600))
9
10 # Hintergrund
11 background = pygame.image.load('background.png')
12
13 # Titel and Icon
14 pygame.display.set_caption("Space Invaders")
15 icon = pygame.image.load('ufo.png')
16 pygame.display.set_icon(icon)
17
18 # Player
19 playerImage = pygame.image.load('space-invaders.png')
20 playerX = 370
21 playerY = 480
22 changeX = 0
23
24 # Enemy
25 enemyImage = pygame.image.load('alien.png')
26 enemyX = random.randint(0,736)
27 enemyY = random.randint(50,150)
28 changeXenemy = 0.3
29 changeYenemy = 40
30
31 def player(x, y):
32     screen.blit(playerImage, (x, y))
33
34 def enemy(x, y):
35     screen.blit(enemyImage, (x, y))
36
37 # Gameloop
38 running = True
39 while running:
40     for event in pygame.event.get():
41         if event.type == pygame.QUIT:
42             running = False
43
44         # uberprufe, ob beim Drucken der Tastatur die Linkspfeiltaste oder
45         # Rechtspfeiltaste gedruckt wird
46         if event.type == pygame.KEYDOWN:
47             if event.key == pygame.K_LEFT:
48                 changeX = -0.3
49             if event.key == pygame.K_RIGHT:
50                 changeX = 0.3
51         if event.type == pygame.KEYUP:
52             if event.key == pygame.K_LEFT or event.key == pygame.K_RIGHT:
53                 changeX = 0
54
55         # RGB
56         screen.fill((0, 0, 0))
57
58         # Background
59         screen.blit(background, (0,0))
60
61         # Anderung der x-Koordinate ubernehmen
62         playerX = playerX + changeX
63
64         # Grenzen uberprufen
65         if playerX <= 0:
66             playerX = 0
67         elif playerX > 736:
68             playerX = 736
69
70
```

```
71 # Bewegung des Gegners
72 enemyX = enemyX + changeXenemy
73 if enemyX <= 0:
74     changeXenemy = 0.3
75     enemyY += changeYenemy
76 elif enemyX > 736:
77     changeXenemy = -0.3
78     enemyY += changeYenemy
79
80 player(playerX, playerY)
81 enemy(enemyX, enemyY)
82 pygame.display.update()
```

4 Schießen

Nun haben wir sowohl einen Spieler (die Rakete), welche wir steuern können, als auch Gegner, welche sich bewegen. Ein wirkliches Spiel haben wir allerdings trotzdem noch nicht, dafür fehlt das eigentliche Ziel des Spiels: Die Gegner abschießen und probieren, nicht Game Over zu gehen. Kümmern wir uns also als nächstes darum, die Gegner abschießen zu können.

4.1 Kugel hinzufügen

Damit der Spieler Gegner abschießen kann, wird eine Kugel implementiert, die aus dem Raumschiff abgefeuert wird. Die Kugel bewegt sich vertikal nach oben und trifft, wenn sie einen Gegner berührt.

```
1 # Bullet
2 bulletImage = pygame.image.load('bullet.png')
3 bulletX = 0
4 bulletY = 480
5 changeXbullet = 0
6 changeYbullet = 0.5
7 bulletState = "ready"
```

Erklärung der Variablen:

- `bulletImage`: Das Bild der Kugel wird mit `pygame.image.load()` geladen.
- `bulletX` und `bulletY`: Die Position der Kugel wird festgelegt. Die Kugel startet auf der Höhe 480, damit sie aus dem Spieler heraus abgefeuert wird.
- `changeXbullet` und `changeYbullet`: Diese Variablen bestimmen die Geschwindigkeit der Kugel. Da sich die Kugel nur nach oben bewegt, ist `changeXbullet` hier 0. Die vertikale Geschwindigkeit beträgt 0.5, was bedeutet, dass sich die Kugel langsam nach oben bewegt.
- `bulletState`: Diese Variable verwaltet den Status der Kugel:
- `"ready"`: Die Kugel ist bereit zum Abschuss, aber aktuell nicht sichtbar.
- `"fire"`: Die Kugel ist abgeschossen und sichtbar.

Das Bild für die Kugel `bullet.png` stammt von der Webseite [Flaticon](#) und kann dort mit der Suchanfrage "bullet" gefunden werden, außerdem sollte es mit einer Auflösung von 32x32 Pixeln heruntergeladen werden.

4.2 Kugel abfeuern

Damit die Kugel tatsächlich abgefeuert werden kann, benötigen wir eine Funktion, die das Abfeuern auslöst und die Kugel auf dem Bildschirm anzeigt.

```
1 def fireBullet(x, y):
2     global bulletState
3     bulletState = "fire"
4     screen.blit(bulletImage, (x + 16, y + 10))
```

Erklärung der Funktion:

- Die Funktion `fireBullet(x, y)` nimmt zwei Parameter entgegen: die X- und Y-Koordinaten der Kugel.
- Mit dem Schlüsselwort `global` wird sichergestellt, dass die Variable `bulletState` innerhalb der Funktion verändert werden kann.
- `bulletState` wird auf `"fire"` gesetzt, sodass das Spiel erkennt, dass eine Kugel gerade aktiv ist und auf dem Bildschirm dargestellt werden muss.
- Mit `screen.blit(bulletImage, (x + 16, y + 10))` wird das Kugelbild an der entsprechenden Position gezeichnet.

Warum wird `x + 16` und `y + 10` verwendet? - Die Kugel soll genau aus der Mitte des Spielers abgefeuert werden. - Falls das Spielerbild beispielsweise eine Breite von 64 Pixeln hat, wird die Kugel mit `x + 16` etwas weiter nach rechts verschoben, sodass sie mittig aus dem Raumschiff kommt. - Die Y-Koordinate wird mit `y + 10` leicht nach unten versetzt, um eine optisch passendere Abschussposition zu erzielen.

Durch diese Funktion wird sichergestellt, dass die Kugel nur dann gezeichnet wird, wenn sie aktiv ist, und dass sie aus der richtigen Position heraus startet.

4.3 Kugel mit der Leertaste abfeuern

Damit der Spieler eine Kugel abfeuern kann, muss das Spiel auf die Leertaste reagieren und die Funktion `fireBullet()` aufrufen.

```

1 ...
2
3 if event.type == pygame.KEYDOWN:
4     if event.key == pygame.K_LEFT:
5         changeX = -0.3
6     if event.key == pygame.K_RIGHT:
7         changeX = 0.3
8     if event.key == pygame.K_SPACE:
9         fireBullet(playerX, bulletY)
10
11 ...

```

Erklärung:

- `event.type == pygame.KEYDOWN` prüft, ob eine Taste gedrückt wurde.
- `event.key == pygame.K_SPACE` stellt sicher, dass es sich um die Leertaste handelt.
- Wenn die Leertaste gedrückt wird, wird `fireBullet(playerX, bulletY)` aufgerufen. Dadurch wird die Kugel aus der aktuellen X-Position des Spielers abgefeuert.

4.4 Bewegung der Kugel

Sobald die Kugel abgefeuert wurde, muss sie sich nach oben bewegen. Dies geschieht mit folgender Logik:

```

1 ...
2
3 # Bewegung der Kugel
4 if bulletState is "fire":
5     fireBullet(playerX, bulletY)
6     bulletY -= changeYbullet

```

Erklärung:

- `if bulletState is "fire"` stellt sicher, dass die Kugel nur dann bewegt wird, wenn sie aktiv ist.
- `fireBullet(playerX, bulletY)` zeichnet die Kugel in jeder Iteration des Spiel-Loops erneut, damit sie sichtbar bleibt.
- `bulletY -= changeYbullet` bewegt die Kugel nach oben, indem ihr Y-Wert kontinuierlich verringert wird.

Da Pygame den Bildschirm bei jeder Aktualisierung neu zeichnet, sorgt diese Logik dafür, dass die Kugel flüssig nach oben fliegt, bis sie entweder einen Gegner trifft oder den oberen Rand des Fensters erreicht.

4.5 Bug fixes

Mit unserer aktuellen Implementierung gibt es leider 2 Probleme. Erstens, kann man immer nur eine Kugel abschießen, bevor man wieder neu starten muss. Zweitens, bewegt sich die Kugel mit dem Raumschiff in X-Koordinaten mit. Probieren wir also diese Bugs zu lösen.

4.5.1 Bugfix: Mehrfaches Abschießen der Kugel ermöglichen

Im ursprünglichen Code bleibt die Kugel, nachdem sie den oberen Bildschirmrand verlassen hat, im Zustand "fire" und kann nicht erneut abgefeuert werden. Um das zu verhindern, müssen wir die Kugel zurücksetzen, wenn sie den Rand erreicht.

```

1 # Bewegung der Kugel
2 if bulletY < -20:
3     bulletY = 480
4     bulletState = "ready"
5
6 if bulletState is "fire":
7     fireBullet(playerX, bulletY)
8     bulletY -= changeYbullet

```

Erklärung:

- `if bulletY < -20`: Sobald die Kugel den oberen Bildschirmrand überschreitet, wird sie zurückgesetzt.
- `bulletY = 480`: Dadurch wird die Kugel auf die Startposition zurückgesetzt (auf Höhe des Spielers).
- `bulletState = "ready"`: Der Zustand der Kugel wird auf "ready" gesetzt, sodass eine neue Kugel abgeschossen werden kann.

- Dadurch kann der Spieler nun mehrere Kugeln nacheinander abfeuern, anstatt nach jedem Schuss das Spiel neustarten zu müssen.

4.5.2 Bugfix: Kugel soll sich nicht mit dem Raumschiff bewegen

Damit sich die Kugel nach dem Abschuss nicht mehr mit dem Spieler bewegt, müssen wir die X-Koordinate der Kugel nur beim Feuern setzen und anschließend beibehalten.

```
1 ...
2
3 if event.key == pygame.K_SPACE:
4     bulletX = playerX
5     fireBullet(bulletX, bulletY)
6
7 ...
8
9 if bulletState is "fire":
10     fireBullet(bulletX, bulletY)
11     bulletY += changeYbullet
```

Erklärung:

- Beim Drücken der Leertaste wird überprüft, ob der `bulletState` auf `"ready"` steht.
- Falls ja, wird `bulletX` auf die aktuelle Position des Spielers gesetzt.
- Danach wird `fireBullet(bulletX, bulletY)` aufgerufen, sodass die Kugel an der richtigen Stelle erscheint.
- **Wichtig:** Danach bleibt `bulletX` gleich, sodass sich die Kugel nicht mehr mit dem Spieler bewegt. - Im unteren Code-Abschnitt bewegt sich die Kugel nur noch vertikal, da `bulletX` nicht mehr verändert wird.

Durch diese Änderungen wird sichergestellt, dass die Kugel immer gerade nach oben fliegt und nicht mehr mit dem Raumschiff mitschwingt.

4.5.3 Bugfix: Verhindern, dass die X-Koordinate beim Feuern verändert wird

Der aktuelle Code erlaubt es, die Leertaste beliebig oft zu drücken, wodurch sich die X-Koordinate der Kugel jedes Mal erneut setzen kann. Dies führt dazu, dass sich die Kugel manchmal falsch positioniert, insbesondere wenn der Spieler sich gleichzeitig bewegt. Um dies zu verhindern, wird überprüft, ob die Kugel bereits aktiv ist, bevor die X-Koordinate geändert wird.

```
1 if event.key == pygame.K_SPACE and bulletState == "ready":
2     bulletX = playerX
3     fireBullet(bulletX, bulletY)
```

Erklärung:

- Die Bedingung `if event.key == pygame.K_SPACE and bulletState == "ready"` stellt sicher, dass die Kugel nur dann abgefeuert wird, wenn sie bereit ist.
- `bulletX = playerX` wird nur ein einziges Mal beim Abschießen gesetzt.
- Danach wird die Funktion `fireBullet(bulletX, bulletY)` aufgerufen, wodurch die Kugel sichtbar wird.
- Sobald die Kugel abgefeuert wurde, bleibt die X-Koordinate unverändert, selbst wenn der Spieler sich während des Fluges bewegt.

Durch diese Änderung wird verhindert, dass sich die X-Position der Kugel während des Fliegens weiter verändert, und das Abschießen der Kugel funktioniert nun exakt so, wie es vorgesehen ist.

4.6 Kollisionserkennung zwischen Kugel und Gegner

Damit der Spieler Gegner besiegen kann, müssen wir überprüfen, ob eine Kugel mit einem Gegner kollidiert. Dafür nutzen wir eine Distanzberechnung basierend auf dem Satz des Pythagoras.

```
1 import math
2
3 ...
4
5 # Score
6 score = 0
```

```

7
8 ...
9
10 def isCollision(enemyX, enemyY, bulletX, bulletY):
11     distance = math.sqrt((math.pow(enemyX - bulletX, 2)) + (math.pow(enemyY - bulletY,
12     2)))
13     if distance < 25:
14         return True
15     else:
16         return False

```

Erklärung der isCollision()-Funktion:

- Diese Funktion überprüft, ob die Kugel den Gegner trifft.
- Sie berechnet die **Distanz** zwischen dem Gegner und der Kugel mit der Formel:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

wobei:

- enemyX, enemyY die Position des Gegners ist,
- bulletX, bulletY die Position der Kugel ist.
- Falls die berechnete Distanz kleiner als 25 Pixel ist, gibt die Funktion **True** zurück, was bedeutet, dass eine Kollision erkannt wurde.
- Ansonsten gibt die Funktion **False** zurück.

Verarbeitung einer Kollision:

Falls eine Kollision erkannt wird, setzen wir die Kugel zurück und erhöhen den Punktestand:

```

1 ...
2 # Kollision
3 collision = isCollision(enemyX, enemyY, bulletX, bulletY)
4 if collision:
5     bulletY = 480
6     bulletState = "ready"
7     score += 1
8     enemyX = random.randint(0, 736)
9     enemyY = random.randint(50, 150)

```

Ablauf der Kollisionserkennung:

1. Der Wert von collision wird durch Aufruf von isCollision(enemyX, enemyY, bulletX, bulletY) berechnet.
2. Falls eine Kollision erkannt wird:
 - Die Kugel wird auf ihre Ausgangsposition (bulletY = 480) zurückgesetzt.
 - Der Kugelstatus wird wieder auf "ready" gesetzt, sodass eine neue Kugel abgeschossen werden kann.
 - Der Punktestand score wird um 1 erhöht.
 - Der Gegner erhält eine neue zufällige Position, um eine neue Herausforderung zu erzeugen.

Diese Logik ermöglicht es, dass der Spieler Punkte sammelt und neue Gegner erscheinen, sobald ein Treffer erfolgt.

Code nach Kapitel 4

```
1 import pygame
2 import random
3 import math
4
5 # wir initialisieren (erstellen) das Spiel.
6 pygame.init()
7
8 # erstellen des Bildschirms
9 screen = pygame.display.set_mode((800,600))
10
11 # Hintergrund
12 background = pygame.image.load('background.png')
13
14 # Titel and Icon
15 pygame.display.set_caption("Space Invaders")
16 icon = pygame.image.load('ufo.png')
17 pygame.display.set_icon(icon)
18
19 # Player
20 playerImage = pygame.image.load('space-invaders.png')
21 playerX = 370
22 playerY = 480
23 changeX = 0
24
25 # Enemy
26 enemyImage = pygame.image.load('alien.png')
27 enemyX = random.randint(0,736)
28 enemyY = random.randint(50,150)
29 changeXenemy = 0.3
30 changeYenemy = 40
31
32 # Bullet
33 bulletImage = pygame.image.load('bullet.png')
34 bulletX = 0
35 bulletY = 480
36 changeXbullet = 0
37 changeYbullet = 0.5
38 bulletState = "ready"
39 # bulletState ready -> man kann die Kugel nicht sehen
40 # bulletState fire -> man kann die Kugel sehen
41
42 # Score
43 score = 0
44
45 def player(x, y):
46     screen.blit(playerImage, (x, y))
47
48 def enemy(x, y):
49     screen.blit(enemyImage, (x, y))
50
51 def fireBullet(x,y):
52     global bulletState
53     bulletState = "fire"
54     screen.blit(bulletImage, (x + 16, y+10))
55
56 def isCollision(enemyX, enemyY, bulletX, bulletY):
57     distance = math.sqrt((math.pow(enemyX-bulletX, 2)) + (math.pow(enemyY-bulletY, 2)))
58     if distance < 25:
59         return True
60     else:
61         return False
62
63 # Gameloop
64 running = True
65 while running:
66     for event in pygame.event.get():
67         if event.type == pygame.QUIT:
68             running = False
69
70     # uberprufe, ob beim Drucken der Tastatur die Linkspfeiltaste oder
71     # Rechtspfeiltaste gedruckt wird
```

```

71     if event.type == pygame.KEYDOWN:
72         if event.key == pygame.K_LEFT:
73             changeX = -0.3
74         if event.key == pygame.K_RIGHT:
75             changeX = 0.3
76         if event.key == pygame.K_SPACE and bulletState is "ready":
77             bulletX = playerX
78             fireBullet(bulletX, bulletY)
79
80     if event.type == pygame.KEYUP:
81         if event.key == pygame.K_LEFT or event.key == pygame.K_RIGHT:
82             changeX = 0
83
84     # RGB
85     screen.fill((0, 0, 0))
86
87     # Background
88     screen.blit(background, (0,0))
89
90     # Anderung der x-Koordinate übernehmen
91     playerX = playerX + changeX
92
93     # Grenzen uberprufen
94     if playerX <= 0:
95         playerX = 0
96     elif playerX > 736:
97         playerX = 736
98
99     # Bewegung des Gegners
100    enemyX = enemyX + changeXenemy
101    if enemyX <= 0:
102        changeXenemy = 0.3
103        enemyY += changeYenemy
104    elif enemyX > 736:
105        changeXenemy = -0.3
106        enemyY += changeYenemy
107
108    # Bewegung der Kugel
109    if bulletY < -20:
110        bulletY = 480
111        bulletState = "ready"
112    if bulletState is "fire":
113        fireBullet(bulletX, bulletY)
114        bulletY -= changeYbullet
115
116    # Kollision
117    collision = isCollision(enemyX, enemyY, bulletX, bulletY)
118    if collision:
119        bulletY = 480
120        bulletState = "ready"
121        score += 1
122        enemyX = random.randint(0,736)
123        enemyY = random.randint(50,150)
124
125    player(playerX, playerY)
126    enemy(enemyX, enemyY)
127    pygame.display.update()

```


5 Verbesserung des Spiels

Unser grundlegendes Spiel ist jetzt eigentlich fertig: Wir können unsere Rakete steuern, wir haben Gegner, die sich bewegen und wir haben die Möglichkeit, unsere Gegner abzuschießen. Das Spiel kann allerdings noch weiter verbessert werden, wie wir anhand folgender Punkte noch sehen werden.

5.1 Mehrere Gegner hinzufügen

Bisher gab es nur einen einzelnen Gegner, der nach jeder Kollision an einer neuen Position erschien. Um das Spiel anspruchsvoller zu machen, werden nun mehrere Gegner gleichzeitig auf dem Spielfeld dargestellt.

```
1 ...
2 # Enemy
3 enemyX = []
4 enemyY = []
5 changeXenemy = []
6 changeYenemy = []
7 NumberOfEnemies = 6
8
9 for i in range(NumberOfEnemies):
10     enemyX.append(random.randint(0,736))
11     enemyY.append(random.randint(50,150))
12     changeXenemy.append(0.3)
13     changeYenemy.append(40)
14
15 enemyImage = pygame.image.load('alien.png')
```

Erklärung:

- NumberOfEnemies = 6: Gibt an, wie viele Gegner gleichzeitig auf dem Spielfeld sein sollen.
- Anstatt einzelne Variablen für enemyX und enemyY zu nutzen, werden nun Listen erstellt, um die Positionen mehrerer Gegner zu speichern.
- Mit einer for-Schleife werden die Startwerte für alle Gegner zufällig gesetzt und in den Listen gespeichert.
- Die Bewegungsgeschwindigkeit jedes Gegners wird ebenfalls in Listen gespeichert (changeXenemy und changeYenemy), sodass sich jeder Gegner unabhängig bewegen kann.

5.2 Bewegung und Kollision für mehrere Gegner

Damit sich alle Gegner unabhängig bewegen und auf Kollisionen mit der Kugel reagieren können, wird die Bewegungslogik in einer Schleife angepasst:

```
1 ...
2 # Bewegung des Gegners
3 for i in range(NumberOfEnemies):
4     enemyX[i] = enemyX[i] + changeXenemy[i]
5     if enemyX[i] <= 0:
6         changeXenemy[i] = 0.3
7         enemyY[i] += changeYenemy[i]
8     elif enemyX[i] > 736:
9         changeXenemy[i] = -0.3
10        enemyY[i] += changeYenemy[i]
11
12 # Kollision
13 collision = isCollision(enemyX[i], enemyY[i], bulletX, bulletY)
14 if collision:
15     bulletY = 480
16     bulletState = "ready"
17     score += 1
18     enemyX[i] = random.randint(0,736)
19     enemyY[i] = random.randint(50,150)
20
21     enemy(enemyX[i], enemyY[i])
```

Erklärung:

- Die Bewegung aller Gegner erfolgt nun innerhalb einer Schleife:
- Jeder Gegner bewegt sich nach rechts oder links basierend auf seinem eigenen changeXenemy[i]-Wert.

- Falls ein Gegner den linken oder rechten Rand erreicht, ändert sich seine Richtung und er rückt nach unten.
- Die Kollisionserkennung läuft ebenfalls innerhalb der Schleife:
- Falls eine Kollision erkannt wird, wird die Kugel zurückgesetzt und der Punktestand erhöht.
- Anschließend erhält der getroffene Gegner eine neue zufällige Position.
- Schließlich wird jeder Gegner mit `enemy(enemyX[i], enemyY[i])` auf dem Bildschirm gezeichnet.

Auswirkungen dieser Änderung:

- Es gibt nun mehrere Gegner gleichzeitig auf dem Spielfeld.
- Jeder Gegner bewegt sich unabhängig und kann einzeln getroffen werden.
- Das Spiel wird dadurch herausfordernder und abwechslungsreicher.

5.3 Score-Anzeige im Spiel

Um den Punktestand während des Spiels sichtbar zu machen, wird eine Funktion erstellt, die den aktuellen Score anzeigt. Dafür nutzen wir die Text-Rendering-Funktionalität von Pygame.

```
1 # Score
2 score = 0
3 font = pygame.font.Font('freesansbold.ttf', 32)
4 textX = 10
5 textY = 10
```

Erklärung:

- `score = 0`: Die Variable hält den aktuellen Punktestand.
- `font = pygame.font.Font('freesansbold.ttf', 32)`:
- Erstellt eine Schriftart für die Anzeige des Scores.
- Hier wird `freesansbold.ttf` verwendet, eine Standard-Schriftart von Pygame.
- Die Zahl 32 gibt die Schriftgröße an.
- `textX` und `textY` legen die Position des Punktestands im Spielfenster fest (oben links bei (10, 10)).

5.4 Funktion zur Score-Anzeige

Damit der Score tatsächlich auf dem Bildschirm erscheint, wird eine Funktion erstellt:

```
1 def showScore(x, y):
2     scorePicture = font.render("Score: " + str(score), True, (255,255,255))
3     screen.blit(scorePicture, (x, y))
```

Erklärung:

- `font.render()` erzeugt eine neue Grafik mit dem aktuellen Punktestand als Text.
- Der erste Parameter `"Score: " + str(score)` setzt den Textinhalt zusammen.
- `True` aktiviert Anti-Aliasing für eine glattere Darstellung.
- `(255,255,255)` gibt die Farbe des Textes an (weiß).
- `screen.blit(scorePicture, (x, y))` zeichnet den Score an der angegebenen Position.

5.5 Score in der Hauptspiel-Schleife anzeigen

Damit der Score ständig sichtbar bleibt, muss die `showScore()`-Funktion in der Spielschleife aufgerufen werden:

```
1 player(playerX, playerY)
2 showScore(textX, textY)
3 pygame.display.update()
```

Anpassung der Schriftart:

Falls eine andere Schriftart gewünscht ist, kann eine eigene `.ttf`-Datei verwendet werden. Kostenlose Schriftarten findet man z. B. auf der Webseite [DaFont](https://dafont.com/). Dort kann man verschiedene Schriftstile herunterladen und in das Spiel einfügen, indem man die Datei ersetzt:

```
1 font = pygame.font.Font('custom_font.ttf', 32)
```

Dadurch lässt sich der Punktestand in einer individuell gestalteten Schriftart darstellen.

5.6 Musik und Soundeffekte

Um das Spielerlebnis zu verbessern, wird das Spiel mit Hintergrundmusik und Soundeffekten ausgestattet. Dazu nutzen wir das `mixer`-Modul von Pygame. Die kostenlosen Sounddateien stammen von [Pixabay](#).

5.6.1 Hintergrundmusik hinzufügen

Die Hintergrundmusik läuft kontinuierlich im Hintergrund, um eine Atmosphäre für das Spiel zu schaffen.

```
1 from pygame import
2
3 # Hintergrundmusik
4 mixer.music.load('background.mp3')
5 mixer.music.play(-1)
```

Erklärung:

- `mixer.music.load('background.mp3')`:
- Lädt die Musikdatei `background.mp3`.
- `mixer.music.play(-1)`:
- Startet die Musik in einer Endlosschleife
(-1 bedeutet, dass die Musik unendlich wiederholt wird).

5.6.2 Soundeffekt beim Schießen

Jedes Mal, wenn der Spieler eine Kugel abfeuert, soll ein passender Soundeffekt abgespielt werden.

```
1 if event.type == pygame.KEYDOWN:
2     if event.key == pygame.K_SPACE and bulletState == "ready":
3         bulletSound = mixer.Sound('shot.mp3')
4         bulletSound.play()
5         bulletX = playerX
6         fireBullet(bulletX, bulletY)
```

Erklärung:

- Die Bedingung `if event.key == pygame.K_SPACE and bulletState == "ready"` stellt sicher, dass der Sound nur beim tatsächlichen Feuern abgespielt wird.
- `mixer.Sound('shot.mp3')` lädt die Datei `shot.mp3`.
- `bulletSound.play()` spielt den Sound ab.
- Danach wird die Kugel wie gewohnt abgefeuert.

5.6.3 Soundeffekt bei einer Explosion

Wenn eine Kugel einen Gegner trifft, wird ein Explosionssound abgespielt:

```
1 if collision:
2     explosionSound = mixer.Sound('explosion.mp3')
3     explosionSound.play()
4     bulletY = 480
5     bulletState = "ready"
6     score += 1
7     enemyX[i] = random.randint(0,736)
8     enemyY[i] = random.randint(50,150)
```

Erklärung:

- Falls eine Kollision erkannt wurde, wird die Datei `explosion.mp3` geladen und abgespielt.
- Danach wird die Kugel zurückgesetzt und ein neuer Gegner erscheint.

5.7 Game Over Anzeige

Damit das Spiel endet, wenn ein Gegner den unteren Bereich des Spielfelds erreicht, wird eine **Game Over**-Nachricht auf dem Bildschirm angezeigt.

5.7.1 Game Over Schriftart initialisieren

Zunächst wird eine größere Schriftart für den Game Over Text definiert:

```
1 # Game Over Text
2 gameOverFont = pygame.font.Font('freesansbold.ttf', 64)
```

Erklärung:

- `pygame.font.Font('freesansbold.ttf', 64)` erstellt eine Schriftart mit der Größe 64.
- Diese wird verwendet, um den **Game Over**-Text gut sichtbar in der Mitte des Bildschirms darzustellen.

5.7.2 Funktion zur Anzeige des Game Over Textes

Eine eigene Funktion kümmert sich darum, den Text im Falle einer Niederlage darzustellen.

```
1 def gameOverText():
2     gameOverPicture = gameOverFont.render("GAME OVER!", True, (255,0,0))
3     screen.blit(gameOverPicture, (200,250))
```

Erklärung:

- `gameOverFont.render("GAME OVER!", True, (255,0,0))` erzeugt eine Grafik mit dem roten **Game Over**-Text.
- `screen.blit(gameOverPicture, (200,250))` zeichnet den Text in der Mitte des Bildschirms bei (200,250).

5.7.3 Game Over Bedingung in der Spielschleife

Nun wird überprüft, ob ein Gegner den unteren Bereich des Spielfelds erreicht hat:

```
1 # Bewegung des Gegners
2 for i in range(NumberOfEnemies):
3
4     # Game Over
5     if enemyY[i] > 420:
6         for j in range(NumberOfEnemies):
7             enemyY[j] = 2000
8             gameOverText()
9             break
```

Erklärung:

- Die Schleife überprüft die Y-Koordinate jedes Gegners.
- Falls ein Gegner unterhalb von 420 Pixeln ankommt, wird das Spiel beendet:
- Eine innere Schleife setzt alle Gegner weit außerhalb des sichtbaren Bereichs (`enemyY[j] = 2000`), sodass sie nicht mehr sichtbar sind.
- Die Funktion `gameOverText()` wird aufgerufen, um die Nachricht **GAME OVER!** anzuzeigen.
- Durch `break` wird die Schleife unterbrochen, sodass keine weiteren Spielprozesse mehr ablaufen.

Code nach Kapitel 5

```
1 import pygame
2 import random
3 import math
4 from pygame import mixer
5
6 # wir initialisieren (erstellen) das Spiel.
7 pygame.init()
8
9 # erstellen des Bildschirms
10 screen = pygame.display.set_mode((800,600))
11
12 # Hintergrund
13 background = pygame.image.load('background.png')
14
15 # Hintergrundmusik
16 mixer.music.load('background.mp3')
17 mixer.music.play(-1)
18
19 # Titel and Icon
20 pygame.display.set_caption("Space Invaders")
21 icon = pygame.image.load('ufo.png')
22 pygame.display.set_icon(icon)
23
24 # Player
25 playerImage = pygame.image.load('space-invaders.png')
26 playerX = 370
27 playerY = 480
28 changeX = 0
29
30 # Enemy
31 enemyX = []
32 enemyY = []
33 changeXenemy = []
34 changeYenemy = []
35 NumberOfEnemies = 6
36
37 for i in range(NumberOfEnemies):
38     enemyX.append(random.randint(0,736))
39     enemyY.append(random.randint(50,150))
40     changeXenemy.append(0.3)
41     changeYenemy.append(40)
42
43 enemyImage = pygame.image.load('alien.png')
44
45 # Bullet
46 bulletImage = pygame.image.load('bullet.png')
47 bulletX = 0
48 bulletY = 480
49 changeXbullet = 0
50 changeYbullet = 0.5
51 bulletState = "ready"
52 # bulletState ready -> man kann die Kugel nicht sehen
53 # bulletState fire -> man kann die Kugel sehen
54
55 # Score
56 score = 0
57 font = pygame.font.Font('freesansbold.ttf', 32)
58 textX = 10
59 textY = 10
60
61 # Game Over Text
62 gameOverFont = pygame.font.Font('freesansbold.ttf', 64)
63
64 def player(x, y):
65     screen.blit(playerImage, (x, y))
66
67 def enemy(x, y):
68     screen.blit(enemyImage, (x, y))
69
70 def fireBullet(x,y):
71     global bulletState
```

```

72     bulletState = "fire"
73     screen.blit(bulletImage, (x + 16, y+10))
74
75 def isCollision(enemyX, enemyY, bulletX, bulletY):
76     distance = math.sqrt((math.pow(enemyX-bulletX, 2)) + (math.pow(enemyY-bulletY, 2)))
77     if distance < 25:
78         return True
79     else:
80         return False
81
82 def showScore(x,y):
83     scorePicture = font.render("Score: " + str(score), True, (255,255,255))
84     screen.blit(scorePicture, (x,y))
85
86 def gameOverText():
87     gameOverPicture = gameOverFont.render("GAME OVER!", True, (255,0,0))
88     screen.blit(gameOverPicture, (200,250))
89
90 # Gameloop
91 running = True
92 while running:
93     for event in pygame.event.get():
94         if event.type == pygame.QUIT:
95             running = False
96
97     # uberprufe, ob beim Drucken der Tastatur die Linkspfeiltaste oder
98     # Rechtspfeiltaste gedruckt wird
99     if event.type == pygame.KEYDOWN:
100         if event.key == pygame.K_LEFT:
101             changeX = -0.3
102         if event.key == pygame.K_RIGHT:
103             changeX = 0.3
104         if event.key == pygame.K_SPACE and bulletState is "ready":
105             bulletSound = mixer.Sound('shot.mp3')
106             bulletSound.play()
107             bulletX = playerX
108             fireBullet(bulletX, bulletY)
109
110         if event.type == pygame.KEYUP:
111             if event.key == pygame.K_LEFT or event.key == pygame.K_RIGHT:
112                 changeX = 0
113
114     # RGB
115     screen.fill((0, 0, 0))
116
117     # Background
118     screen.blit(background, (0,0))
119
120     # Anderung der x-Koordinate ubernehmen
121     playerX = playerX + changeX
122
123     # Grenzen uberprufen
124     if playerX <= 0:
125         playerX = 0
126     elif playerX > 736:
127         playerX = 736
128
129     # Bewegung des Gegners
130     for i in range(NumberOfEnemies):
131
132         # Gameover
133         if enemyY[i] > 420:
134             for j in range(NumberOfEnemies):
135                 enemyY[j] = 2000
136                 gameOverText()
137                 break
138
139         enemyX[i] = enemyX[i] + changeXenemy[i]
140         if enemyX[i] <= 0:
141             changeXenemy[i] = 0.3
142             enemyY[i] += changeYenemy[i]
143         elif enemyX[i] > 736:
144             changeXenemy[i] = -0.3

```

```

144         enemyY[i] += changeYenemy[i]
145
146     # Kollision
147     collision = isCollision(enemyX[i], enemyY[i], bulletX, bulletY)
148     if collision:
149         explosionSound = mixer.Sound('explosion.mp3')
150         explosionSound.play()
151         bulletY = 480
152         bulletState = "ready"
153         score += 1
154         enemyX[i] = random.randint(0,736)
155         enemyY[i] = random.randint(50,150)
156
157     enemy(enemyX[i], enemyY[i])
158
159     # Bewegung der Kugel
160     if bulletY < -20:
161         bulletY = 480
162         bulletState = "ready"
163     if bulletState is "fire":
164         fireBullet(bulletX, bulletY)
165         bulletY -= changeYbullet
166
167
168     player(playerX, playerY)
169     showScore(textX, textY)
170     pygame.display.update()

```

6 Game Over und Restart-Funktion

Bisher hatte unser Spiel keine Möglichkeit, nach einem Game Over neu zu starten. Dafür fügen wir nun eine Restart-Funktion hinzu, die das Spiel zurücksetzt, wenn die Leertaste (SPACE) gedrückt wird.

6.1 Game Over Status hinzufügen

Zunächst benötigen wir eine Variable, die speichert, ob das Spiel vorbei ist.

```
1 # Game Over Text
2 gameOverFont = pygame.font.Font("freesansbold.ttf", 64)
3 gameOverState = False
```

- **gameOverState:** Speichert, ob das Spiel vorbei ist (True bedeutet Game Over).

6.2 Game Reset-Funktion

Damit das Spiel von vorne beginnt, setzen wir alle Variablen auf ihre Anfangswerte zurück.

```
1 #Reset Game
2 def resetGame():
3     global playerX, playerY, changeX, enemyX, enemyY, changeXenemy, changeYenemy
4     global NumberOfEnemies, bulletX, bulletY, changeXbullet, changeYbullet
5     global bulletState, gameOverState, score
6
7     # Reset playerposition
8     playerX = 370
9     playerY = 480
10    changeX = 0
11
12    # Replace enemies
13    enemyX = []
14    enemyY = []
15    changeXenemy = []
16    changeYenemy = []
17    NumberOfEnemies = 8
18
19    for i in range(NumberOfEnemies):
20        enemyX.append(random.randint(0,736))
21        enemyY.append(random.randint(50,150))
22        changeXenemy.append(0.3)
23        changeYenemy.append(40)
24
25    # Reset gameOverState
26    gameOverState = False
27
28    # Reset bullet
29    bulletX = 0
30    bulletY = 480
31    changeXbullet = 0
32    changeYbullet = 0.5
33    bulletState = "ready"
34
35    # Reset score
36    score = 0
```

- **Alle Spielvariablen** werden auf ihre Ausgangswerte zurückgesetzt.
- **Gegner** werden neu platziert diesmal mit 8 statt 6 Aliens.
- **gameOverState** wird wieder auf False gesetzt, sodass das Spiel neu startet.
- **Score und Kugelzustand** werden zurückgesetzt, damit die Punkte von null beginnen.

6.3 Restart einleiten

Nun erlauben wir dem Spieler, das Spiel durch Drücken von SPACE neuzustarten, wenn sich das Spiel im GameState befindet.

```
1 # Neustart nach Game Over
2 if event.key == pygame.K_SPACE and gameState == True:
3     resetGame()
```

Highscore hinzufügen

Ein Highscore-System speichert die höchste erreichte Punktzahl und zeigt sie an. Dazu fügen wir eine **Highscore-Variable**, eine **Anzeige-Funktion** und die **Aktualisierung der Highscore-Werte** hinzu.

6.4 Highscore-Variable

Zunächst definieren wir die Variable highscore, die den höchsten erreichten Punktestand speichert.

```
1 # Highscore-Variable
2 highscore = 0
```

- **highscore = 0:** Initialisiert die Highscore-Variable mit 0.

6.5 Highscore-Anzeige

Nun erstellen wir eine Funktion, die den Highscore auf dem Bildschirm anzeigt.

```
1 # Highscore anzeigen
2 def showHighScore(x, y):
3     scorePicture = font.render("HighScore: " + str(highscore), True, (255,255,255))
4     screen.blit(scorePicture, (x, y))
```

- **font.render(...):** Erstellt ein Textbild für den Highscore.
- **screen.blit(...):** Zeichnet den Highscore-Text auf den Bildschirm.
- **x, y:** Definieren die Position der Anzeige.

6.6 Highscore in der Game Loop aktualisieren

Die Highscore-Variable muss aktualisiert werden, wenn der aktuelle Score höher ist als der bisherige Highscore.

```
1 # Highscore aktualisieren
2 if score > highscore:
3     highscore = score
```

- Falls der aktuelle Punktestand (score) größer als der gespeicherte highscore ist, wird der Highscore aktualisiert.

6.7 Highscore auf dem Bildschirm anzeigen

Nachdem der Highscore aktualisiert wurde, muss die Anzeige-Funktion aufgerufen werden.

```
1 # Highscore anzeigen
2 showHighScore(10, 45)
```

- **showHighScore(10, 45):** es wird die showHighScore-Funktion aufgerufen und der Highscore wird auf die Koordinaten (10, 45) gesetzt